

Reinforcement Learning

Dynamic Programming

Dynamic Programming (DP) is a **model-full method** — it requires **complete knowledge of the environment** (transition probabilities, rewards) to compute optimal value/policy functions using the Bellman equations.

DP breaks down the large RL problem into **optimal subproblems**, similar to how DP works in DSA — solving the largest problem by solving the smallest sub-problems first.

RL Solution Methods — Overview

Method	Description
1. Dynamic Programming (DP)	Model-full — needs complete env knowledge
2. Monte Carlo Methods	Model-free — learns from episodes
3. TD Learning	Model-free — learns from each step

Bellman Equation (DP Core)

The Bellman equation computes the **Value Function $V(s)$** — the best expected return starting from state s :

$$V(s) = \max_a (R(s, a) + \gamma \cdot V(s'))$$

where $R(s,a)$ is the immediate reward, γ is the discount factor, and $V(s')$ is the value of the next state.

Disadvantages of Dynamic Programming

- 1

Model-full
Requires complete knowledge of the environment — impractical for most real-world problems.
- 2

Scalability
Does not scale well to large or complex state spaces.
- 3

Computationally Expensive
Very costly for large MDPs with millions of states.

Reinforcement Learning

Monte Carlo Methods

Monte Carlo (MC) Methods are **model-free algorithms** — they do NOT need complete knowledge of the environment. They learn from **experienced samples (episodes)**, computing returns from full episode trajectories.

How MC Works — Episode Example

Consider a 3×3 grid where S=(2,0) is start and E=(0,2) is exit. After one episode ($\gamma=1$):

Step	State	Action	Reward
1	(2,0)	↑	-1
2	(1,0)	↑	-1
3	(0,0)	→	-1
4	(0,1)	→	0 (exit)

Return G_t from step 1 ($\gamma=1$):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} = -1 + (-1) + (-1) = -3$$

So $V(2,0) = -3$ for episode 1. After episode 2, $V(2,0) = -4$. Average: $V(2,0) = (-3 + -4)/2 = -3.5$

MC Updation Step

After running multiple episodes, $V(s)$ is updated as the **average of returns** from all episodes that visited that state:

$$V(s) = \text{average of } G_t \text{ across all episodes visiting } s$$

Limitations of Monte Carlo

1 Episode-based — Slow Learning

Requires complete episodes to update. Long episodes lead to slow learning.

2 Continuous Problems

Cannot be applied to continuous (non-terminating) problems — MCM requires finite episodes.

3 Learning = Avg of episode consequences

Learns the average consequences of individual paths, not each step individually.

Note: MC learns after each full episode; TD Learning (next topic) learns after every single step — making it much faster.

Reinforcement Learning

Temporal Difference (TD) Learning

TD Learning is the most important among the 3 RL methods and is the central idea in modern RL. It is the closest to how humans learn. It is a **model-free RL method** that combines the ideas of DP and Monte Carlo.

Unlike Monte Carlo where updates happen after the full episode, in TD Learning **updates happen after each step**. This enables faster learning and faster convergence.

TD Update Rule

$$V(s) = V(s) + \alpha \cdot [R + \gamma \cdot V(s') - V(s)]$$

Symbol	Meaning
α (alpha)	Learning rate
R	Reward received
$\gamma \cdot V(s')$	Estimated value of next state
$V(s)$	Current state value
$[R + \gamma V(s') - V(s)]$	TD Error — how wrong our assumption was

The **TD Error** shows how wrong our current estimate is. Our aim is to make this TD error closer to **0**, so $V(s)$ converges to the optimal value.

TD Learning — Worked Example ($\alpha=0.5$, $\gamma=1$)

Grid: 3×3 , $S=(2,0)$, $E=(0,2)$. All $V(s)$ initialized to 0.

Step 1: Agent at $(2,0)$, takes action $\uparrow$, gets reward = -1, next state = $(1,0)$

$$V(2,0) = V(2,0) + 0.5 \times [-1 + 1 \cdot V(1,0) - V(2,0)] = 0 + 0.5 \times (-1) = -0.5$$

Step 2: Agent at $(1,0)$, takes action $\uparrow$, reward = -1, next state = $(0,0)$

$$V(1,0) = V(1,0) + 0.5 \times [-1 + 1 \cdot V(0,0) - V(1,0)] = 0 + 0.5 \times (-1) = -0.5$$

TD Algorithms

1 SARSA

On-policy TD algorithm — stepwise learning.

2 Q-Learning

Off-policy TD algorithm — faster learning, faster convergence.

Reinforcement Learning

SARSA

SARSA stands for **State, Action, Reward, Next State, Next Action**. It is an **on-policy TD algorithm** that learns the Q-value (Action-Value function) for each state-action pair.

It is model-free, uses Q-values (Action Values), and belongs to the TD Methods family. Goal: **maximize total rewards** by learning to act in the environment.

SARSA Steps

1 Current State (S_t)

Observe the current state.

2 Choose Action (A_t)

Use ϵ -greedy policy to select an action. (1000 episodes $\rightarrow$ 10% explore, 90% exploit)

3 Take Action

Execute the action $\rightarrow$ receive Reward R_{t+1} and Next State S_{t+1} .

4 Next Action (A_{t+1})

Select next action using policy: $\text{Policy}(S_{t+1}, A_{t+1})$.

5 Update Q-value

Apply the SARSA update rule:

$$Q(s,a) = Q(s,a) + \alpha [R + \gamma \cdot Q(s', a') - Q(s, a)]$$

where α = learning rate, γ = discount factor, a' = next action chosen by policy.

Note: SARSA is called **on-policy** because it uses the actual next action a' selected by the current policy for the update — it follows a safer path.

Reinforcement Learning

Q-Learning

Q-Learning is an **off-policy TD algorithm**. Unlike SARSA, it always picks the **next best action** for the update, regardless of what action the agent actually takes — making it more aggressive and finding the **shortest path** to the goal.

Q-Learning Steps

1 Current State (S_t)

Observe the current state.

2 Choose Action (A_t)

Select action using ϵ -greedy policy.

3 Take Action

Execute action $\rightarrow$ receive Reward R_{t+1} , observe Next State S_{t+1} .

4 Next Action (A_{t+1})

On basis of Q-value — pick the **next best action** for the next state (not the actual taken action).

5 Update Q-value

Apply the Q-Learning update rule:

$$Q(s,a) = Q(s,a) + \alpha \cdot [R + \gamma \cdot \max_a Q(s', a) - Q(s, a)]$$

The key difference: **$\max Q(s',a)$** — always picks the highest Q-value for next state, regardless of policy $\rightarrow$ optimal Q-value.

SARSA vs Q-Learning Comparison

Aspect	SARSA	Q-Learning
Policy Type	On-policy	Off-policy
Next Action	From current policy	Best possible (max Q)
Path Found	Safer path	Shortest/optimal path
Speed	Standard	Faster convergence
Update uses	$Q(s', a')$ actual action	$\max Q(s', a)$ best action

Cliff Walking Problem

Classic RL benchmark. Environment: **4×12 grid**.

Element	Definition
State (S)	$4 \times 12 = 48$ cells (row, col) $\rightarrow$ single num: $\text{row} \times 12 + \text{col}$
Action (A)	up(0), down(1), left(2), right(3) $\rightarrow \{0,3\}$
Reward	Normal cells $\rightarrow -1$ Target E $\rightarrow 0$ Cliff $\rightarrow -100$

Start S	$(3,0) \rightarrow \text{state} = 3 \times 12 + 0 = 36$
Q-table size	48 states $\times$ 4 actions

Deep Reinforcement Learning

RL + DL = DQN

Standard Q-Learning uses a **Q-table** $Q(s,a)$ — works only for small, discrete state spaces (e.g., 48 states). It fails when:

- There are **millions of states** → Q-table becomes huge and impractical.
- State values are **continuous** → impossible to enumerate.

Solution: **Deep Q-Networks (DQN)** — replace the Q-table with a Neural Network that approximates Q-values directly from states.

$$Q\text{-Learning} \rightarrow Q(s, a) \mid DQN \rightarrow Q(s, a; \theta) \text{ where } \theta = \text{NN parameters}$$

DQN Architecture

Input: state vector (e.g., 48 nodes) → Hidden Layers → Output: Q-value for each action (e.g., 4 actions). Training uses **Forward Pass (FP)**, **Loss**, **Backpropagation (BP)**.

Layer	Description
Input Layer	State representation (48 nodes for cliff walking)
Hidden Layer	1+ layers, 256 nodes, ReLU activation
Output Layer	Q-value for each action (4 nodes)

DQN Challenges & Solutions

Problem 1: Correlated Samples

Naive DQN feeds sequential experiences $E1 \rightarrow E2 \rightarrow E3 \rightarrow E4$ into the NN, but NNs require **independent samples**. Sequential experiences are highly correlated.

Solution: Experience Replay (Replay Buffer)

Store all experiences in a replay buffer (deque), then **randomly sample a mini-batch** (e.g., size=16) for training. This breaks correlations.

$$\text{experience} = (\text{State}, \text{Action}, \text{Reward}, \text{Next State}, \text{Terminated})$$

Deque (Double-Ended Queue): add/remove from both ends. Used in FIFO mode (First In First Out). Old experiences are removed as new ones are added.

Problem 2: Moving Target

The target y in Q-learning update is not a fixed value — it changes as the network weights update, causing unstable training.

$$\text{Target: } y = R + \gamma \cdot \max_a Q(s', a; \theta)$$

$$\text{Predicted Q-value: } Q(s, a; \theta) \rightarrow \blacksquare$$

$$\text{Loss: } (y - \blacksquare)^2$$

Solution: Target Network — DQN uses **2 neural networks**:

- **Policy Network (online)** — updated every step

- **Target Network (old copy of policy)** — updated every N steps (e.g., 10 steps), providing a stable target

Note: Both networks have the same architecture but different weights. The target network provides a *FIXED* target for a period, stabilizing training.

Deep Q-Networks

Training Flappy Bird (Minor Project)

Applying DQN to train a Flappy Bird agent — a Reinforcement Learning minor project. The bird (agent) learns to navigate through pipes using trial and error.

Actions: 0 → do nothing | 1 → flap (jump)

DQN Setup Steps

0 Set Environment

Configure the Flappy Bird gym environment using LIDAR sensor inputs.

1 Define DQN

Neural network architecture: Input=12 nodes, Hidden=256 nodes (1 layer), Output=2 nodes (actions).

2 Create Experience Replay

Initialize deque replay buffer to store (State, Action, Reward, NextState, Terminated) tuples.

3 Multiple Episodes — DQN Train Setup

Run indefinitely until stopping condition (manual Ctrl+C or reward threshold).

4 ϵ -Greedy Policy + Hyperparams

Epsilon starts at 1.0 and decays gradually: 1.0 → 0.9 → 0.8 → 0.7 → ... → 0.05 (epsilon decay).

5 Target Network

Policy Network updates every step; Target Network updates every 10 steps.

6 Train & Test

run() function: Train mode and Test & Visualize/Render mode.

Device Setup

Device selection for GPU acceleration:

- Apple (M1, M2, M3) → **MPS** (Metal Performance Shaders)
- Nvidia → **CUDA** toolkit for GPU training
- Default fallback → **CPU**

Gymnasium Environment API

Gymnasium (gym) creates environment objects with these key methods:

Method	Purpose
make()	Create the environment
render()	Show / visualize the environment
reset()	Reset to initial state
close()	Close the environment

step()	Take an action in the environment
--------	-----------------------------------

GenAI

Generative AI

Generative AI (GenAI) refers to AI systems that can **create new content** — text, images, audio, video, code — that resembles real data. It is a subset of Deep Learning.

AI Hierarchy	Models Used
AI → ML → DL → NLP	RNN, LSTM (text/sequence tasks)
AI → ML → DL → CV	CNN (image/video tasks)
GenAI → LLMs	Transformers (text generation)
GenAI → Image Gen	CNN, Diffusion, GAN

GenAI Applications

Content Type	Tools / Models
Images / Videos	DALL-E, Midjourney
Text	ChatGPT, Claude, Gemini
Audio / Sound / Music	SoundDraw
Code	GitHub Copilot

LLMs

Large Language Models

LLMs are a **class of models in NLP** (Natural Language Processing) that work with text. They are a specific type of GenAI powered by **Transformer Architecture**.

Key Characteristics:

- Billions of parameters
- Trained on huge datasets
- Continuously improving capabilities

LLM Examples

GenAI → LLMs → Transformer Architecture

LLM	Company	Architecture
ChatGPT / GPT-4	OpenAI	Decoder-only Transformer
Claude	Anthropic	Decoder-only Transformer
Gemini	Google	Decoder-only Transformer
BERT	Google	Encoder-only Transformer

Transformers

Introduced in **2017** by Google Brain in the paper "*Attention is All You Need*". A Transformer is a **Neural Network architecture** designed for sequential/text data.

Before Transformers: **RNN, LSTM, CNN** processed sequences **step by step (sequentially)** — called sequence models. Transformers use **Attention** to process all tokens **in parallel** → faster training + better performance.

Word Embeddings

Embedding

Text must be converted to numbers (vectors) before feeding into a neural network. Each word is mapped to a vector of numbers:

"I like Python" → Tokens: [I, like, Python]

I → [0.1, 0.7] | like → [0.32, 0.41] | Python → [0.88, 0.2]

Each dimension of the vector represents a **feature** (e.g., is it a language? is it an object?). GPT-4 uses **12,288-dimensional embeddings** per token.

Static vs Contextual Embedding

Static (Word2Vec): Word2Vec uses a 2-layer NN to produce static embeddings — same vector regardless of context.

"book" → [0.32, 0.5, 0.01] (same for "read a book" AND "book tickets!")

Problem: 'Python' → snake? or programming language? Static embedding can't distinguish by context.

Contextual Embedding (Transformers): The same word gets **different vectors depending on context** — this is achieved through the Attention mechanism inside the Transformer encoder.

Transformer Architecture

The Transformer has two main parts: the **Encoder** (understands) and the **Decoder** (generates). Each consists of N stacked layers.

Component	Role	Examples
Encoder	Understand input sequence → contextual embeddings	BERT (encoder-only)
Decoder	Generate output one token at a time	GPT / Claude (decoder-only)
Full Transformer	Translate/summarize (seq-to-seq)	T5, original Transformer

Applications of Encoder: Summarization, Sentiment Analysis, Text Classification, Speech Translation.

Encoder

Positional Encoding

Since Transformers process all tokens in parallel (not sequentially like RNN), they lose track of **word order**. Positional Encoding (PE) adds position information to the embeddings.

RNN processes: I → like → Python (step by step, knows order)

Transformer processes: I, like, Python all at once → must encode position separately

Final output = Word Embedding + Positional Embedding

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d_{model}})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{2i/d_{model}})$$

Parallelization → **faster training** and **higher performance** compared to sequential RNN processing.

Attention

Attention is All You Need

Attention mechanism tells each word in the sequence **what other words to focus on**. Example: "The animal didn't cross the street because it was tired." — what does "it" refer to? The animal! Attention resolves this.

Attention uses **3 matrices** derived from the input:

Matrix	Meaning	Dimension
Q (Query)	What I am looking for	$n \times d_k$
K (Key)	What I can offer	$n \times d_k$
V (Value)	The actual information content	$n \times d_v$

These are computed as: $Q = X \cdot W_Q$ | $K = X \cdot W_K$ | $V = X \cdot W_V$ (W = learned weight matrices)

Scaled Dot-Product Attention

1 Similarity Score

Compute dot product of Q and K^T :

$$Q \cdot K^T$$

2 Scaling

Divide by $\sqrt{d_k}$ to prevent vanishing gradients:

$$Q \cdot K^T / \sqrt{d_k}$$

3 Softmax

Apply softmax to get probabilities (attention weights):

$$\text{softmax}(Q \cdot K^T / \sqrt{d_k})$$

4 Attention Output

Multiply weights by V:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Multi-Head Attention

Instead of one attention head, use **multiple heads in parallel** — each learning a different aspect (grammar, meaning, references). Outputs are concatenated and projected.

3 Types of Attention:

Type	Used In	Purpose
Self-Attention	Encoder	Each token attends to all tokens in same sequence
Cross-Attention	Decoder	Decoder attends to encoder output
Masked Attention	Decoder training	Prevents future tokens from being seen (no cheating)

Residual Connection + Normalization

Each sub-layer in the encoder uses **Add + Norm**: input is directly connected to the output (residual connection), then normalized.

Solves: vanishing gradient + distorted information in deep NNs → **preserves original information**

Normalization: faster convergence, stabilizes training.

Feed-Forward Network (FNN)

After attention, each token passes through a position-wise FNN. It adds **non-linearity** and increases model complexity. Functions: ① gather information ② think deeply about the information

Result → more enriched, contextual embeddings. GPT-2: N=48 layers | GPT-3: N=96 layers

Decoder

Cross-Attention & Masked Attention

The Decoder generates output **one token at a time**. It receives contextual embeddings from the Encoder and generates the output sequence.

Cross-Attention: Attention between two different sequences — decoder attends to encoder's contextual embeddings to generate each output token.

Masked Attention: During training, the decoder can see the full target sequence (which is 'cheating'). Masked attention prevents future tokens from being seen → forces the model to predict the next token without peeking.

BERT vs GPT Architectures

Feature	BERT	GPT / Claude
Architecture	Encoder-only	Decoder-only
Goal	Understand text	Generate text
Processing	Bidirectional	Left-to-right (token by token)
Training	Easier to train	Easier to fine-tune
Use Cases	Classification, Sentiment	Conversation, Text Generation

Pre-trained Transformers

Pre-trained Transformers are already trained on **huge datasets for general tasks** (SOTA AI models). They can be **fine-tuned** on smaller specific datasets.

Key pre-trained models: BERT, GPT-3, T5 (Text-to-Text Transfer Transformer)

Benefits of fine-tuning (self-training):

1 Small dataset → better generalization

Avoids overfitting while achieving strong performance.

2 Time saving

No need to train from scratch — saves weeks of compute.

3 Money saving

Billions of parameters already trained — reuse cheaply.

Platforms for Pre-trained Models: Hugging Face, PyTorch Hub, TensorFlow Hub, OpenAI APIs

Generative Adversarial Networks (GANs)

Architecture

GANs generate data that **looks like real data**. They consist of **2 Neural Networks** competing against each other:

- **Generator (G)** — takes random noise → generates fake data
- **Discriminator (D)** — classifies input as real (1) or fake (0)

noise (z) → G → fake data → D → classification (0/1)

D also receives real data — it must distinguish real from fake. As G improves, D is fooled more often. As D improves, G must generate better fakes. This adversarial training continues until G produces realistic data.

Applications of GANs:

- Rare dataset augmentation for training
- Ghibli-style art generation (artistic filters)
- Deepfakes

Generator Network

Vanilla GAN (fully connected / dense): $y = Wz + b$

Noise $z \rightarrow$ Dense layers with UPSAMPLING (expansion) $\rightarrow$ Image (RGB $64 \times 64 \times 3$)

DCGAN (Deep Convolutional GAN):

noise(z) $\rightarrow$ Dense(fully connected) $\rightarrow$ Reshape (feature map $8 \times 1 \times 128$) $\rightarrow$ ConvTranspose layers $\rightarrow$ $16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64 \times 3$ (RGB image)

Noise types: Gaussian ($\mu=0$, $\text{std}=1$) or Uniform distribution, dimension = 100

Discriminator Network

Vanilla D: $64 \times 64 \times 3$ image $\rightarrow$ Flatten $\rightarrow$ Linear layers ($1024 \rightarrow 512 \rightarrow 256$) $\rightarrow$ Sigmoid $\rightarrow$ 0/1 output (Binary Cross Entropy loss, CNN)

G $\rightarrow$ **upsampling** $\rightarrow$ generates | D $\rightarrow$ **downsampling** $\rightarrow$ classifies

GAN Loss Functions

1 Generator Loss (minimize)

G wants D to classify its fakes as real ($D(G(z)) \rightarrow 1$):

$$g\text{-loss} = -\log(D(G(z)))$$

2 Discriminator Loss (minimize)

D wants to correctly classify real as 1 and fake as 0:

$$d\text{-loss} = -[\log(D(x)) + \log(1 - D(G(z)))]$$

Note: GANs have limitations: training is unstable, and it is hard to evaluate quality. This led to the development of **Diffusion Models** (e.g., Stable Diffusion) as a more stable alternative.

Retrieval-Augmented Generation (RAG)

Traditional LLMs are trained on fixed training data — they suffer from: ① **Hallucination** (making things up) ② **Not up-to-date** information. RAG solves this by combining Information Retrieval with LLM generation.

RAG improves LLM output:

- Up-to-date information from external sources
- Higher accuracy — grounded in retrieved facts
- Fact-based generation

Real-world example: Perplexity AI

RAG Architecture — 3 Phases

Phase 1: Data Ingestion Pipeline

Convert knowledge base documents into a vector database:

Step	Tool/Process
1. Load Data (pdf/html/db/excel)	Document Loaders (LangChain)
2. Process/Clean → Chunks	Text Splitter
3. Embed Chunks → Vectors	OpenAI Embedding / Sentence Transformer
4. Store in Vector DB	Pinecone / FAISS / ChromaDB

Document object has 2 fields: ① page_content (text) ② metadata

Phase 2: Retrieval Pipeline

When user queries, retrieve relevant context from vector DB:

Step	Process
4. Query Processing	Embed the user query using same embedding model
5. Similarity Search	Vector DB semantic search → top K results (cosine similarity)
6. Augmentation	Combine top K context + original query → augmented prompt
7. Generation	Send augmented prompt to LLM → generate output

cosine similarity = 1 – cosine distance (vectors, not text-based)

Phase 3: Generation

LLM generates the final answer using both the retrieved context and the user query — grounded, accurate, and up-to-date.

LangChain Implementation

LangChain provides high-level abstractions for building RAG pipelines:

Component	LangChain Class
-----------	-----------------

Document Loaders	PyPDFLoader, CSVLoader, etc.
Text Splitter	RecursiveCharacterTextSplitter
Embeddings	OpenAIEmbeddings / HuggingFaceEmbeddings
Vector Store	Chroma, FAISS, Pinecone
Retriever	VectorStoreRetriever → returns list of context docs
LLM	ChatOpenAI / HuggingFacePipeline

| *Keep Learning & Keep Exploring!*